Stoquify migration issues: diagnosis and permanent resolution route

Date: 2026-08-28 Scope: Prisma/PostgreSQL migration history, destructive-change assurance, bootstrap/adoption paths, schema evidence, and release automation Decision status: **local history healthy; production migration control remains blocked and requires remediation**

Executive explanation

The database migration system is not generally broken. The configured local database is currently healthy: a live read-only query found all 79 repository migrations successfully applied, with no unfinished or rolled-back rows, no unknown or duplicate successful migrations, and no checksum mismatch.

The lasting issue is the migration **control plane**, not the current local schema. Production release logic scans all historical SQL and blocks on 13 destructive operations inside an old baseline-only bridge. That is correct fail-closed behavior, but the supporting approval/evidence workflow is incomplete, stale, and coupled to mutable application files. As a result, the project can repeatedly prove that the local database is fine and still remain unable to produce a durable production-deployment decision.

The best route is not a reset, a migration rewrite, or a blanket approval. It is to separate:

1. **full-history integrity** — prove that the repository and target migration histories are immutable and coherent; from
2. **target-pending risk** — review only the migrations that a verified target would actually execute.

This keeps old risk visible without forcing every future release to reapprove a historical migration that the target already applied with the correct checksum. If the baseline is still pending, the gate remains strict and selects one of two controlled paths: execute only on a proven empty database, or manually resolve as applied on a verified compatible existing database.

What is healthy now

Control	Current evidence	Decision
Repository catalog	79 migration directories; every directory has a non-empty `migration.sql` and conforms to the expected name pattern	Healthy
Configured local target	79/79 completed rows; 0 unfinished; 0 rolled back; 0 missing; 0 checksum mismatches; 0 unknown; 0 duplicate successes	Healthy for this local target only
Approval registries	Both JSON registries parse and contain zero entries	Structurally healthy, no decisions recorded
Static production wiring	Migration preflight is ordered before application build; policy and release chains include migration gates	Mechanism exists
Baseline guard	The bridge is transactional and refuses execution when target schema/data indicators are present	Useful defense, not sufficient authorization
Earlier seed/schema reconciliation	The 64-migration state passed schema, seed, credential, and POS verification on 2026-08-15	Valid historical evidence, no longer current for 79 migrations

No production or remote database was inspected. Current local health must not be promoted into a production claim.

Confirmed issues

1. The historical baseline bridge contains real destructive operations

`prisma/migrations/20260611130000_accounting_auth_baseline_bridge/migration.sql` contains:

- 12 `DROP COLUMN` operations across `accounts`, `sessions`, and `users`;
- one `DROP TABLE auth_sessions` operation; and
- no data-copy/backfill for several renamed or replacement auth fields.

The affected data includes OAuth tokens, provider identity, session tokens and expiry, email-verification state, and session-revocation provenance. The SQL therefore cannot be approved merely because it is old.

The migration itself clearly defines two contexts:

- **Existing database:** verify schema and data, then use `prisma migrate resolve --applied`; never execute the bridge.
- **Genuinely empty database:** execute the chain only after proving the destructive boundary contains no meaningful rows and the complete chain reproduces the intended schema.

The current static gate reports 13 findings, zero approvals, and one blocker: `destructive_sql_is_exact_hash_approved`.

2. The evidence bundle is incomplete and structurally brittle

The live evidence gate reports:

- status `REJECTED_BLOCKED_EVIDENCE_INCOMPLETE`;
- 0/14 completed required artifacts;
- no maker identity or independent checker decision;
- no exact-hash approval;
- a rolling development candidate captured while dirty;
- changed Git HEAD/tree since capture; and
- bound-file mismatches in `package.json`, `prisma/schema.prisma`, and the settings users client.

Some of those failures are correct. Missing backup, restore, affected-data, auth-regression, and independent approval evidence must block production.

The design flaw is that a historical SQL approval is also invalidated by normal evolution of mutable files such as the current Prisma schema, application consumers, package metadata, or Git HEAD. A durable approval should bind the immutable migration and its reviewed evidence. A separate release manifest should bind the current source commit and current consumer state. Combining the two creates perpetual evidence churn.

3. The destructive gate evaluates the whole catalog rather than the target's pending set

`scripts/prisma-production-migration-gate.js` scans every migration file and requires every destructive finding in the repository to be approved. It does not first query the selected target's successful migration history and

derive which migrations will execute.

Consequences:

- a target that already applied the baseline correctly remains blocked by its historical SQL;
- an empty target and an existing mature target receive the same destructive-risk decision even though their actions differ;
- the system cannot distinguish audit visibility from deploy authorization; and
- old risk becomes a permanent release blocker rather than a target-specific execution control.

The repair is a two-gate model: full-history integrity over the entire catalog, followed by destructive review over the exact pending set.

4. Fresh-replay evidence is behind the live catalog

The saved fresh-replay certificate covers 62/62 migrations. The later schema/seed reconciliation covers 64 migrations. The repository now contains 79.

These reports are not false; they are historical. They cannot prove that a brand-new database can currently replay the complete chain. A new isolated 79/79 replay, second-run no-op, and schema diff are required before the empty-database path is trusted.

5. Environment-specific history problems existed, but they are not present on the current local target

An earlier 2026-08-09 report found five missing migrations and a checksum mismatch for `20260619120000_backfill_purchase_receive_permission`. A later report could not query a configured database. The current read-only local query now passes all nine history checks at 79/79.

This means the earlier issue was repaired or belonged to a different local state. It should not be described as a current repository checksum defect. It does prove that environment-specific migration history can diverge, so every controlled environment needs its own current census before release.

6. Two migrations share one timestamp prefix

The following full names are unique but share `20260818120000`:

- `20260818120000_governed_delivery_order_to_cash`
- `20260818120000_pos_electronic_tender_authority`

Prisma orders the full names deterministically, so this is not a demonstrated application failure. It is an operational ambiguity for tools, reports, or approvals that identify migrations by the timestamp alone.

Do not rename them after application. Grandfather these exact names and add a CI rule that prevents any new duplicate 14-digit prefix.

7. The migration evidence tools use two hash identities

The old maker-checker packet binds the raw migration bytes:

`f7de8dc7ace0a5e063cdb7ecd9527e77807b7f872e474546819e9fca13a273d4`

The production risk gate normalizes CRLF to LF before hashing:

`2fde92f4ad0cbb1a2a517b12e71c38997dc4956c9d57d70ff1ae0c636ff2f191`

Both hashes describe the same current SQL under different canonicalization rules. This does not by itself corrupt a migration, and the history-health gate already accepts raw/LF/CRLF variants. However, approval packets and registries need one versioned canonical identity plus an explicit raw transport hash so a reviewer can prove that all gates refer to the same artifact.

8. Prisma dependency versions are untidy but not the proven migration cause

The installed Prisma CLI and client are 6.19.3. `@prisma/adapter-pg` is 7.9.1 and is not imported by repository application code. This major-version split should be removed or aligned during dependency hygiene, but current evidence does not show it caused the history blocker.

9. Current command verification had an execution limitation

Live read-only JavaScript checks completed and proved the 79/79 local history and current gate status. Attempts to run the full Prisma validation and focused Jest group did not complete within the 30-second command window and left high-CPU child processes, which were terminated. Earlier saved evidence records those checks as passing for smaller catalogs, but a fresh focused test run is still required after implementation. This limitation is not presented as a product defect without further diagnosis.

Architecture impact

The application graph reinforces why the baseline cannot be treated as a database-only cleanup. In `graphify-out/GRAPH_REPORT_app.md`, `FinanceRouteAccess()` is a 14-edge hub in Community 1, while accounting workflows appear in Communities 8, 17, and 20. The destructive packet also binds Better Auth, session assurance, RBAC, security settings, and accounting consumers. Migration decisions therefore affect authentication continuity, finance routes, ledger workflows, and operator access together.

The graph is supporting impact evidence. Migration truth still comes from SQL, `_prisma_migrations`, schema diff, data reconciliation, and recovery rehearsal.

Permanent resolution route

Phase 0 — Freeze and census

1. Freeze existing migration files: no edits, renames, reordering, deletion, or squashing.
2. Add a versioned migration catalog manifest with full name, raw hash, normalized LF hash, and optional CRLF hash.
3. Record every controlled environment using a redacted identifier, owner, target class, history health, baseline state, backup/PITR state, and intended action.
4. Grandfather the two current timestamp collisions; fail on new collisions.

Exit gate: every environment is classified and no unresolved checksum/unknown/unfinished history exists.

Phase 1 — Split history integrity from pending execution risk

Implement this sequence:

```
classify target without leaking credentials
  -> query migration history read-only
  -> verify complete historical integrity
  -> calculate exact pending migration names
  -> scan destructive SQL only in the pending set
  -> require path-specific evidence and approval
  -> deploy
  -> verify history and schema again
```

Full-history checks remain fail-closed for:

- missing catalog;
- applied-file checksum changes;
- unknown successful migrations;
- unfinished/rolled-back state requiring resolution;
- duplicate successful rows;
- invalid, stale, expired, or revoked approvals; and
- unauthorized catalog mutation.

Pending-risk checks remain fail-closed for every destructive operation a target would execute.

Exit gate: a healthy target with zero pending migrations passes deploy preflight without approving historical SQL, while a target with pending destructive SQL stays blocked.

Phase 2 — Encode the two baseline paths

Path A: existing compatible database

1. Capture redacted target identity and history.
2. Prove the post-baseline schema objects already exist.
3. Profile every destructive source field/table and reconcile retained data.
4. Create an encrypted backup/PITR reference and complete a separate restore test.
5. Rehearse `prisma migrate resolve --applied` against an isolated restored clone.
6. Prove application-table counts/checksums and auth/accounting controls are unchanged; only migration metadata may change.
7. Obtain named maker and independent checker approval.
8. Perform the manual resolve in a controlled window. Never automate it in ordinary deploys.

Path B: genuinely empty database

1. Create a disposable isolated database.
2. Prove the auth/accounting objects affected by the bridge contain zero meaningful rows at the destructive boundary.
3. Replay all 79 migrations.
4. Run deploy again and prove no-op idempotency.
5. Diff the physical schema against `prisma/schema.prisma` and classify only explicit allowed residual metadata.
6. Inject a failure and prove transactional rollback/no partial DDL.

7. Run auth/session, RBAC, accounting, seed, and representative transaction smoke checks.
8. Complete backup/restore evidence and independent approval.

If a database matches neither path, stop and create additive expand/backfill/verify/contract migrations. Never force the baseline bridge.

Exit gate: every environment has exactly one approved path or a documented additive-remediation plan.

Phase 3 — Repair evidence binding

1. Version the canonical hash algorithm.
2. Use normalized LF UTF-8 SQL as the approval identity and retain raw SHA-256 as transport integrity.
3. Bind approval to the migration SQL, individual destructive findings, target path, evidence-bundle digest, decision, reviewer independence, time, expiry, and revocation.
4. Snapshot mutable consumer/schema/package hashes into the evidence bundle, but do not use later unrelated changes to invalidate a historical approval.
5. Create a per-release manifest that binds the current Git commit, target environment, exact pending set, referenced approvals, and post-deploy results.

Exit gate: the same migration receives an unambiguous identity in every gate, and later application changes do not erase a valid historical decision.

Phase 4 — Rehearse, approve, and release

Release order:

1. secret and target preflight;
2. full-history health;
3. pending-set calculation;
4. destructive pending-risk decision;
5. backup/PITR attestation;
6. migration deploy;
7. post-deploy history health and `prisma migrate status`;
8. schema drift classification;
9. auth/session and accounting smoke checks; and
10. immutable evidence archive.

Do not put a human approval into the repository until the technical evidence is complete and an independent checker explicitly chooses `APPROVE_EXACT_HASH`.

Exit gate: the selected production target passes the full sequence with no secret leakage, no unapproved pending risk, no post-deploy drift, and no unresolved auth/accounting regression.

Phase 5 — Prevent recurrence

- New schema changes use expand/backfill/verify/contract; destructive cleanup is a later release.
- Applied migration edits fail CI.
- New duplicate timestamps fail CI.
- Blank replay runs on a schedule and on migration changes.
- A representative upgraded snapshot replay runs on high-risk migration changes.

- Every release stores pre/post history, pending set, migration duration, locks/timeouts, schema fingerprint, and smoke outcomes.
- Approval and exception records have owners, review dates, and revocation support.
- Remove or align the unused Prisma 7 adapter dependency separately.

Exit gate: a new migration cannot merge without immutable history proof, current replay evidence, and the correct risk review.

Why this route is preferable

Option	Benefit	Main risk	Decision
Reset databases and recreate history	Fast locally	Data loss, destroys audit/history truth, unacceptable for controlled environments	Reject
Rewrite or rename the old baseline migration	Makes SQL look cleaner	Breaks applied checksums and environment history	Reject
Add blanket approvals for all 13 findings	Unblocks the static gate	No proof of data preservation, restore, or reviewer independence	Reject
Squash all 79 migrations immediately	Shorter new bootstrap	High-risk cutover; complicates existing histories and evidence	Defer
Keep the global all-history destructive gate forever	Strongly fail-closed	Historical risk blocks every release regardless of target state	Replace
Full-history integrity plus target-pending risk	Preserves audit truth and enforces actual execution risk	Requires careful gate refactor and target history query	**Recommended**

Immediate squashing is not the best repair. Once the control plane is stable and production histories are inventoried, a future migration-epoch design can be evaluated for new installations. It should be a separate, evidence-heavy program, not a shortcut around the present blocker.

Focused success criteria

- The current 79/79 local result remains intact.
- Every controlled environment has a current redacted history record.
- Historical applied risk remains visible but does not block a target with no pending migration and matching history.
- Pending destructive SQL always requires exact evidence-bound human approval.
- Existing populated databases cannot execute the baseline bridge automatically.
- A fresh isolated database passes a current 79/79 replay, second-run no-op, drift check, recovery rehearsal, and auth/accounting smoke.
- Both hash identities are reproducible and mapped through one versioned contract.
- New timestamp collisions and applied-file mutation fail CI.
- No production, accounting, security, privacy, or legal certification is claimed without independent evidence and authority.

Evidence consulted

- `prisma/migrations/20260611130000_accounting_auth_baseline_bridge/migration.sql`
- `prisma/migration-risk-approvals.json`
- `prisma/migration-history-checksum-approvals.json`
- `scripts/prisma-production-migration-gate.js`
- `scripts/prisma-migration-history-health-check.js`
- `scripts/prisma-fresh-replay-certification.js`
- `scripts/prisma-destructive-migration-evidence-gate.js`
- `docs/operations/runbooks/prisma-production-migrations.md`
- `docs/reconcile-destructive-migration/EVIDENCE_BUNDLE_STATUS.md`
- `docs/blockers/stoquify-migration-risk-maker-checker-packet-2026-08-13.json`
- `what-next/prisma-migration-deployment-readiness.{md,json}`
- `what-next/prisma-migration-history-health-referral-final.{md,json}`
- `what-next/prisma-fresh-replay-certification-referral.{md,json}`
- `what-next/database-schema-migration-seed-reconciliation-2026-08-15.md`
- `docs/system-evaluation-and-redirection/system-evaluation-2026-08-28-comprehensive-report.md`
- `graphify-out/GRAPH_REPORT_app.md`

Verification record for this diagnosis

Check	Result
Repository migration inventory	PASS — 79 well-formed, non-empty migration directories
Live local history query	PASS — 9/9 checks; 79/79; no history findings
Live static destructive-risk scan	EXPECTED BLOCK — 13 findings, 0 approved
Live evidence-bundle evaluation	EXPECTED BLOCK — incomplete evidence, mutable binding drift, no approval
Duplicate timestamp scan	FINDING — one duplicated prefix across two full migration names
Hash reproduction	PASS — raw and normalized-LF hashes reproduced
Prisma/package version inventory	PASS — versions recorded; adapter not used
`npm run prisma:validate`	INCONCLUSIVE — did not complete inside command window; earlier saved evidence passed
Focused Jest migration group	INCONCLUSIVE — did not complete inside command window; child processes terminated
Remote/production database inspection	NOT RUN — outside current authority and evidence scope